Cloud-Based Exam Question Bank with AI-Assisted Exam Generation

Technical Progress Report | Step 1: Foundation (sub-steps 1 and 2)

4	13	8	36/36
database tables 2 migrations	API operations in 9 route files	secondary indexes 6 foreign keys	end-to-end API checks passing

Item	Detail
Report date	19 October 2026
Repository	`github.com/sannnntf123-pixel/mycapstone`
Stack	Next.js 15.5 (App Router, TypeScript) | PostgreSQL 16 | Prisma 6.19 | JWT (jose) | Zod
Users	Instructors only. Instructors author and approve questions and (in a later step) generate exams.
Status	Project setup, schema, migrations and the full Step-1 REST API are complete and verified. Seed data, automated test suite, app Dockerfile and README are next.

1. Executive summary

This report documents the first delivered increment of the capstone: the back-end foundation for a question bank used by **instructors**. Instructors author exam questions; every question passes an explicit instructor approval step before it enters the bank; and each question can be kept private or shared with other instructors. Later increments will add AI question generation and AI exam assembly, both for instructors. The schema already carries the fields those features need, so they can be added without restructuring the data model.

Scope change (19 Oct 2026): the system no longer has a Student role. Student exam-taking was removed from the plan and the role was dropped from the schema with a dedicated migration (Section 4.6). The system is instructor-only, and every endpoint except registration and login requires an authenticated instructor.

The implementation is a Next.js 15 application exposing a JSON REST API through App Router route handlers, backed by PostgreSQL through the Prisma ORM. Authentication is stateless (signed JWT bearer tokens) and gated by an invite code; authorization for questions is row-level: the visibility rule is defined in exactly one function and every question query is composed with it. All 36 end-to-end checks pass against the running server and a real database.

1.1 Scope and status

#	Sub-step (Step 1)	Status	Evidence
1	Project setup: Next.js, Prisma, PostgreSQL in Docker, schema and initial migration	**Done**	Migration 20261016091130_init
2	REST API: auth, curricula/chapters, questions (create, list/filter, get, approve, reject, bank)	**Done**	36/36 live checks; tsc + eslint clean
2b	Scope change: remove Student role, instructor-only system	**Done**	Migration 20261019100000_remove_student_role
3	Guard review / hardening (DB CHECK on difficulty, login rate limit)	**Next**	Guards in src/lib/auth.ts
4	Seed data (instructors, 2 curricula with chapters, questions in each state)	To do	-
5	Automated test suite (approval + privacy) on an isolated test DB	To do	scripts/smoke-test.mjs to be ported
6	App Dockerfile, full docker-compose, README	To do	DB compose file exists

Out of scope for this step: AI question generation, exam generation and any user interface. Student exam-taking has been removed from the project.

2. Technology stack

Layer	Choice (version)	Rationale
Runtime / framework	Next.js 15.5.28, React 19.1, TypeScript 5, Node 23 (dev)	One codebase serves the API now and the UI later. Route handlers are standard Web Request/Response functions, deployable to Vercel (serverless) or any container host.
Database	PostgreSQL 16 (postgres:16-alpine)	Data is strongly relational (curriculum > chapter > question > user). Native enum types, JSON columns, mature managed offerings on every cloud (RDS, Cloud SQL, Azure, Neon, Supabase).
ORM / migrations	Prisma 6.19.3	Declarative schema is both documentation and source of truth; versioned SQL migrations and a fully typed client, so a typo in a column name is a compile error.
Authentication	jose 6.2 (JWT HS256), bcryptjs 3.0 (cost 10)	Stateless tokens scale horizontally with no session store. jose runs on both the Node and Edge runtimes. Chosen over NextAuth because its Credentials provider makes server-side RBAC and API testing awkward.
Validation	Zod 3.25	Every body and query string is parsed at the boundary; types are inferred from schemas, so validation and TypeScript types cannot drift apart.
Containers	Docker via Colima, docker-compose	Reproducible database locally; same image family used in the cloud.

3. Architecture

The code is organised in strict layers. Each layer only calls the one beneath it, which keeps route handlers trivial and concentrates every business rule in a domain service that can be unit-tested without HTTP.

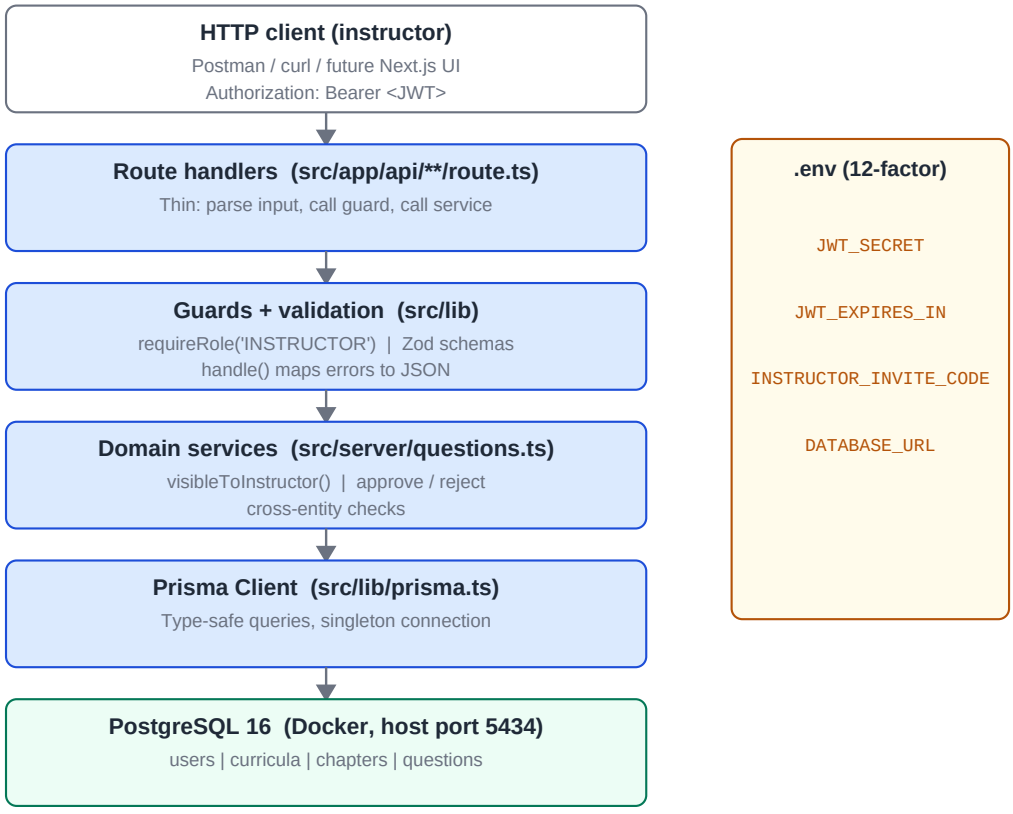

Figure 1 - Layered architecture and request path

3.1 Folder structure

```
my-capstone/
|-- prisma/
|   |-- schema.prisma                data model (source of truth)
|   `-- migrations/                  20261016091130_init, 20261019100000_remove_student_role
|-- src/
|   |-- app/api/                     HTTP layer - thin route handlers
|   |   |-- auth/register|login|me/route.ts
|   |   |-- curricula/route.ts, curricula/[id]/chapters/route.ts
|   |   `-- questions/route.ts, bank/route.ts,
|   |       [id]/route.ts, [id]/approve/route.ts, [id]/reject/route.ts
|   |-- lib/                         cross-cutting infrastructure
|   |   |-- prisma.ts                singleton PrismaClient
|   |   |-- auth.ts                  hashing, JWT, requireUser / requireRole
|   |   |-- errors.ts                HttpError, handle() wrapper
|   |   `-- validation.ts            Zod schemas for every input
|   `-- server/questions.ts          privacy filter + approval workflow
|-- scripts/smoke-test.mjs           end-to-end API checks (npm run smoke)
|-- docs/                            this report + its generator
|-- docker-compose.yml   .env.example   PROCESS.md
```

3.2 Request lifecycle

- The route handler is wrapped in `handle()`, so any thrown error becomes a JSON response with the right status code.
- `requireRole(req, 'INSTRUCTOR')` verifies the bearer JWT signature and expiry, then **reloads the user from the database** and checks the role.
- The request body or query string is parsed by a Zod schema; failure produces HTTP 400 with per-field messages.
- The domain service applies business rules (visibility, chapter-belongs-to-curriculum, status transitions) and queries via Prisma.
- The handler returns `NextResponse.json(...)` with 200/201.

Extension points for later steps: `src/server/ai.ts`, `src/server/exams.ts` and `src/app/api/exams/**` plug in at the same layers; UI pages go under `src/app/`.

4. Database design

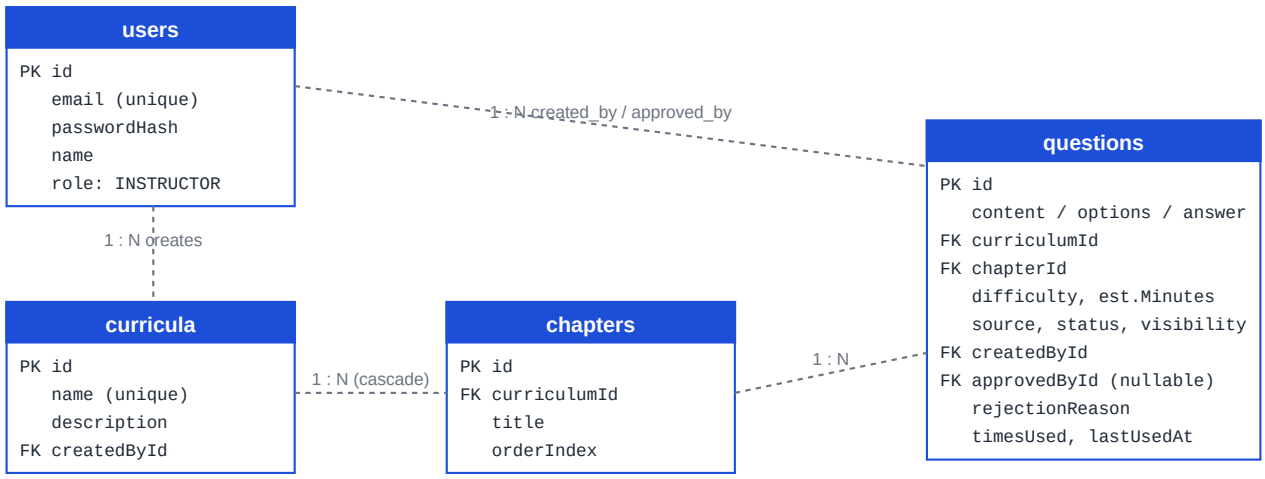

Figure 2 - Entity-relationship overview

Primary keys are `cuid` strings rather than auto-increment integers: they are non-guessable (no enumeration of other instructors' questions by incrementing an id), can be generated without a database round-trip, and are safe to merge across environments. Table names are mapped to snake-case plurals via `@@map`. All tables carry

`createdAt` and an automatically maintained `updatedAt`.

4.1 Enumerated types (native PostgreSQL ENUMs)

Type	Values	Used by
Role	INSTRUCTOR	users.role (default INSTRUCTOR). Kept as an enum so further roles (e.g. Admin) can be added by migration without touching the auth layer.
QuestionStatus	PENDING, APPROVED, REJECTED	questions.status (default PENDING)
Visibility	PRIVATE, SHARED	questions.visibility (default PRIVATE)
QuestionSource	MANUAL, AI	questions.source (default MANUAL; AI reserved for a later step)

4.2 Table: questions

Column	Type	Null	Notes
id	text (cuid)	no	Primary key
content	text	no	Question stem
options	jsonb	yes	Array of choices for MCQ; null for open questions. API enforces answer is one of them.
answer	text	no	Correct answer / model solution
curriculumId	text FK	no	-> curricula.id, ON DELETE RESTRICT
chapterId	text FK	no	-> chapters.id, ON DELETE RESTRICT; must belong to curriculumId (service check)
difficulty	integer	no	1-5 (validated by Zod)
estimatedTimeMinutes	integer	no	1-600 (validated by Zod); input for exam-duration planning
source	QuestionSource	no	Forced to MANUAL by the create endpoint
createdById	text FK	no	-> users.id, ON DELETE RESTRICT; the owner
status	QuestionStatus	no	Forced to PENDING on create
approvedById	text FK	yes	-> users.id, ON DELETE SET NULL; set only on approval
approvedAt	timestamp(3)	yes	Set only on approval
rejectionReason	text	yes	Optional reviewer comment on rejection
visibility	Visibility	no	PRIVATE by default (privacy-safe default)
timesUsed	integer	no	Default 0; incremented by exam generation later
lastUsedAt	timestamp(3)	yes	Lets exam generation prefer less recently used questions
createdAt / updatedAt	timestamp(3)	no	Audit timestamps

4.3 Tables: users, curricula, chapters

Table	Columns	Constraints
users	id, email, passwordHash, name, role, createdAt, updatedAt	email UNIQUE. Only a bcrypt hash is stored, never the password.
curricula	id, name, description?, createdById, createdAt, updatedAt	name UNIQUE; createdById -> users (RESTRICT)
chapters	id, curriculumId, title, orderIndex, createdAt, updatedAt	UNIQUE(curriculumId, orderIndex), UNIQUE(curriculumId, title); curriculumId -> curricula ON DELETE CASCADE

4.4 Indexes and why each exists

Index	Columns	Serves
questions_curriculumId_chapterId_idx	(curriculumId, chapterId)	Filtering by curriculum and chapter; the primary access path for exam assembly
questions_status_visibility_idx	(status, visibility)	Bank queries (status = APPROVED) combined with the shared-visibility predicate
questions_createdById_idx	(createdById)	The 'own questions' branch of the privacy filter and ?mine=true
questions_difficulty_idx	(difficulty)	Difficulty filtering / difficulty-balanced exam generation
users_email_key	UNIQUE(email)	Login lookup and duplicate-registration prevention
curricula_name_key	UNIQUE(name)	Prevents duplicate curricula
chapters_..._orderIndex_key / _title_key	UNIQUE per curriculum	Stable chapter ordering; no duplicate chapter titles

4.5 Referential-integrity decisions

- **Deleting a curriculum cascades to its chapters**, but is **blocked** (RESTRICT) while any question references it, so approved bank content can never disappear silently.
- **Deleting a user is blocked** while they own questions or curricula. If an approver is removed, `approvedById` becomes NULL (SET NULL) while `approvedAt` keeps the historical fact.
- Denormalising `curriculumId` onto questions (it is derivable through the chapter) is deliberate: it makes the most common filter a single-column index lookup with no join.

4.6 Migration history

Migration	Change
20261016091130_init	Creates the four tables, four enum types, eight secondary indexes and six foreign keys.
20261019100000_remove_student_role	Deletes accounts with role STUDENT, then recreates the Role enum with INSTRUCTOR only (PostgreSQL cannot drop an enum value in place) and sets the default to INSTRUCTOR. Safe: students could never own curricula or questions, so no rows are orphaned.

```
DELETE FROM "users" WHERE "role" = 'STUDENT';
CREATE TYPE "Role_new" AS ENUM ('INSTRUCTOR');
ALTER TABLE "users" ALTER COLUMN "role" DROP DEFAULT;
ALTER TABLE "users" ALTER COLUMN "role" TYPE "Role_new" USING ("role"::text::"Role_new");
ALTER TYPE "Role" RENAME TO "Role_old";
ALTER TYPE "Role_new" RENAME TO "Role";
DROP TYPE "Role_old";
ALTER TABLE "users" ALTER COLUMN "role" SET DEFAULT 'INSTRUCTOR';
```

After applying it, `prisma migrate diff` between the live database and `schema.prisma` produced an empty migration, confirming the database and schema are in sync.

5. Authentication and access control

5.1 Registration and login

- Passwords are hashed with **bcrypt, cost factor 10** (salted, deliberately slow); minimum length 8.
- **Every registration requires an invite code** equal to the `INSTRUCTOR_INVITE_CODE` environment variable (missing code: 400, wrong code: 403). Without this, anyone on the internet could create an account and read the shared question bank. If the variable is not set, registration fails closed.
- The role is **always set server-side** to INSTRUCTOR; a `role` field sent by the client is ignored.

- Login returns the same *'Invalid email or password'* (401) for an unknown email and a wrong password, preventing account enumeration.
- Emails are normalised to lower case before storage and lookup.

5.2 Token design

Property	Value
Format / algorithm	JWT, HS256 (HMAC-SHA256) signed with JWT_SECRET (random 256-bit, generated with openssl)
Claims	`sub` = user id, `role`, `iat`, `exp`
Lifetime	JWT_EXPIRES_IN, default 8h (a teaching day)
Transport	`Authorization: Bearer <token>` header (no cookies, therefore no CSRF surface for the API)
Verification	Signature and expiry checked by jose.jwtVerify; then the user row is reloaded

Why reload the user on every request? The token's `role` claim is informational only; authorization uses the role read from the database. A deleted account or a changed role therefore takes effect on the next request instead of when the token expires. The cost is one primary-key lookup per request.

```
export async function requireRole(req: Request, ...roles: Role[]) {
  const user = await requireUser(req);                // 401 if missing/invalid/expired
  if (!roles.includes(user.role)) throw forbidden();   // 403 if wrong role
  return user;
}
```

Every protected route calls `requireRole(req, 'INSTRUCTOR')` explicitly, even though INSTRUCTOR is currently the only role. This keeps each endpoint's permission visible in its own code and means that adding a new role later cannot silently grant it access to existing endpoints.

5.3 Permission matrix

Operation	Anonymous	Instructor
Register (with invite code) / login	yes	-
GET /api/auth/me	no (401)	**yes**
List curricula and chapters	no (401)	**yes**
Create curriculum / chapter	no (401)	**yes**
Create question	no (401)	**yes**
List / filter / get questions	no (401)	own + shared only
View question bank	no (401)	approved, own + shared
Approve / reject	no (401)	pending questions visible to them
Generate exams (later step)	no (401)	planned

6. Privacy model (row-level authorization)

Rule: an instructor may see a question if and only if they created it, or its visibility is SHARED. The rule lives in a single function:

```
export function visibleToInstructor(userId: string): Prisma.QuestionWhereInput {
  return { OR: [{ createdById: userId }, { visibility: "SHARED" }] };
}

// every query composes the rule with the caller's filters using AND:
const where = { AND: [ visibleToInstructor(user.id),
                       { curriculumId, chapterId, difficulty, status, visibility, ... } ] };
```

Because the rule is combined with `AND`, no client-supplied filter can widen visibility. For example, `?visibility=PRIVATE` returns only the caller's own private questions, never another instructor's (verified in Section 11). The rule is applied inside the database query rather than by filtering results in memory, so hidden rows never leave the database and pagination counts stay correct.

Enforcement point	Mechanism
GET /api/questions, /api/questions/bank	listQuestions() composes visibleToInstructor() into findMany and count
GET /api/questions/:id	getVisibleQuestion(): findFirst with id AND visibility rule; else 404
approve / reject	Both call getVisibleQuestion() first, so you cannot review what you cannot see
Anonymous callers	requireRole(req, 'INSTRUCTOR') on every route; 401 before any query runs

404 rather than 403 is returned for another instructor's private question, so the API does not even confirm that the question exists.

7. Approval workflow

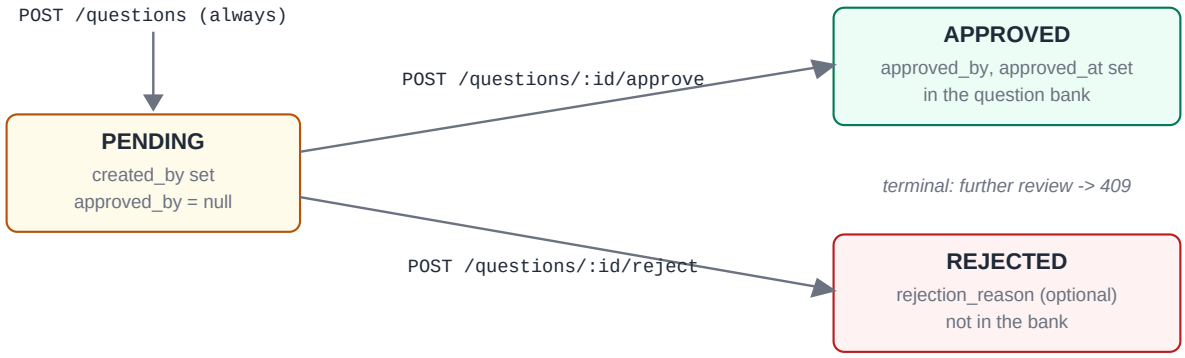

Figure 3 - Question status state machine

Transition	Preconditions	Fields written
create -> PENDING	Instructor; chapter belongs to curriculum; MCQ answer within options	status=PENDING, source=MANUAL, createdById=caller (all server-controlled, client cannot override)
PENDING -> APPROVED	Instructor who can see the question; status still PENDING	approvedById=caller, approvedAt=now(), rejectionReason=null
PENDING -> REJECTED	Same as approve; optional reason	rejectionReason=reason
Reviewed -> any	Not allowed (APPROVED and REJECTED are final)	HTTP 409 Conflict

7.1 Who may approve

Any instructor who can see the question. Shared questions therefore get peer review from any colleague; a private question can only be approved by its owner (nobody else can see it). Whether an owner should be allowed to approve their *own shared* question is an open design question (Section 12).

7.2 Concurrency safety

Two instructors could press 'approve' at the same moment. A naive read-check-write would let both succeed and the last write would overwrite `approvedById`. The update is instead made **conditional on the current status**, which PostgreSQL evaluates atomically:

```
const result = await prisma.question.updateMany({
  where: { id, status: "PENDING" },     // compare-and-set
  data,
});
if (result.count === 0) throw conflict("Question was reviewed by someone else");   // 409
```

7.3 The question bank

The bank is defined as **status = APPROVED**, intersected with the visibility rule. `GET /api/questions/bank` forces `status: 'APPROVED'` server-side, overriding any status in the query string; this is the endpoint the exam generator will draw from in a later step.

8. Validation and error handling

Schema	Rules
registerSchema	email valid + lower-cased; password >= 8; name non-empty; inviteCode required (non-empty)
createQuestionSchema	content, answer non-empty; options: array of >= 2 non-empty strings (optional); difficulty int 1-5; estimatedTimeMinutes int 1-600; visibility PRIVATE|SHARED (default PRIVATE)
listQuestionsQuery	query strings coerced to numbers; status/visibility enums; page >= 1; pageSize 1-100 (default 20)
createChapterSchema	title non-empty; orderIndex positive int
rejectQuestionSchema	reason optional; body itself optional

Unknown fields such as `role`, `status` or `createdById` are stripped by Zod, so clients cannot mass-assign protected fields (verified in Section 11).

Status	Meaning in this API
200 / 201	Success / resource created
400	Validation failed (with per-field details) or inconsistent input (e.g. chapter not in curriculum)
401	Missing, invalid or expired token; bad login
403	Wrong role; invalid invite code
404	Not found, or not visible to the caller
409	Duplicate (email, curriculum, chapter) or invalid status transition / concurrent review
500	Unexpected error; logged server-side, generic message to client (no stack traces leak)

```
HTTP/1.1 400 Bad Request
{ "error": "Validation failed",
  "details": { "difficulty": ["Number must be less than or equal to 5"] } }
```

9. API reference

Method	Path	Access	Description
POST	/api/auth/register	public + invite	Create instructor account; returns { user, token } (201)
POST	/api/auth/login	public	Returns { user, token }
GET	/api/auth/me	instructor	Current user
GET	/api/curricula	instructor	All curricula with ordered chapters
POST	/api/curricula	instructor	Create curriculum { name, description? }
GET	/api/curricula/:id/chapters	instructor	Chapters of a curriculum, ordered
POST	/api/curricula/:id/chapters	instructor	Create chapter { title, orderIndex }
GET	/api/questions	instructor	List visible questions; filters + pagination
POST	/api/questions	instructor	Create question (PENDING)
GET	/api/questions/bank	instructor	Approved visible questions only
GET	/api/questions/:id	instructor	Single visible question
POST	/api/questions/:id/approve	instructor	Approve a pending question
POST	/api/questions/:id/reject	instructor	Reject; optional { reason }

List filters: `curriculumId, chapterId, difficulty, status, visibility, mine=true, page, pageSize`. Results are ordered newest first and returned as `{ items, total, page, pageSize }`.

9.1 Example: register and create a question

```
POST /api/auth/register
{ "email": "alice@uni.edu", "password": "********", "name": "Alice", "inviteCode": "<code>" }
201 Created  { "user": { "id": "cmg...", "role": "INSTRUCTOR", ... },
               "token": "eyJhbGciOiJIUzI1NiJ9..." }

POST /api/questions
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
{ "content": "What is 1 + 1?", "options": ["1","2","3"], "answer": "2",
  "curriculumId": "cmg...", "chapterId": "cmg...", "difficulty": 1,
  "estimatedTimeMinutes": 2, "visibility": "SHARED" }

201 Created   (abridged)
{ "id": "cmg...", "status": "PENDING", "source": "MANUAL", "visibility": "SHARED",
  "approvedBy": null, "approvedAt": null, "timesUsed": 0,
  "chapter": { "title": "Algebra", "orderIndex": 1 }, "createdBy": { "name": "Alice" } }
```

9.2 Example: approve

```
POST /api/questions/cmg.../approve          -> 200 { ..., "status": "APPROVED",
                                                     "approvedBy": { "name": "Bob" },
                                                     "approvedAt": "<ISO-8601 timestamp>" }
POST /api/questions/cmg.../approve (again)  -> 409 { "error": "Question is already approved" }
```

10. Configuration and environment

Variable	Purpose	Example / default
DATABASE_URL	PostgreSQL connection string	`postgresql://capstone:***@localhost:5434/question_bank`
JWT_SECRET	HMAC key for signing tokens	random, from `openssl rand -base64 32`
JWT_EXPIRES_IN	Token lifetime	8h
INSTRUCTOR_INVITE_CODE	Required to register; unset = registration closed	set per deployment
DB_PORT, POSTGRES_*	docker-compose overrides	5434, capstone, question_bank

All configuration is read from the environment (12-factor), so the same build runs locally and in the cloud with different values. `.env` is git-ignored; `.env.example` is committed as the template.

npm script	Command
dev	next dev --turbopack -p 3100
db:up / db:down	docker-compose up -d / docker-compose down
db:migrate	prisma migrate dev
db:studio	prisma studio (visual DB browser)
smoke	node scripts/smoke-test.mjs (end-to-end checks against a running server)
typecheck / lint	tsc --noEmit / eslint

```
npm install
cp .env.example .env          # set JWT_SECRET and INSTRUCTOR_INVITE_CODE
npm run db:up                 # Postgres on localhost:5434
npm run db:migrate            # apply migrations, generate client
npm run dev                   # API on http://localhost:3100
npm run smoke                 # in a second terminal
```

11. Verification

Static checks: `tsc --noEmit` and `eslint` report zero errors; `prisma migrate diff` shows no drift between database and schema. **Dynamic checks:** `scripts/smoke-test.mjs` registers fresh instructors and exercises every endpoint over HTTP against the running server and the real PostgreSQL database. Result after the instructor-only change: **36 of 36 checks passed**.

Area	Check	Result
Auth	Register without invite code -> 400; wrong invite code -> 403	**PASS**
Auth	Client-supplied role is ignored; account is INSTRUCTOR	**PASS**
Auth	Register two instructors; duplicate email -> 409	**PASS**
Auth	Login returns token; wrong password -> 401	**PASS**
Auth	/auth/me returns current user; garbage token -> 401	**PASS**
Access	No token on /auth/me, /curricula, /questions, /questions/bank -> 401 (4 checks)	**PASS**
Curricula	Create curriculum; duplicate name -> 409	**PASS**
Curricula	Create chapter; other instructor can list; no token -> 401	**PASS**
Workflow	New question is PENDING and PRIVATE by default; source MANUAL	**PASS**
Validation	Client cannot set status or owner on create	**PASS**
Validation	difficulty 7 -> 400; chapter from another curriculum -> 400; MCQ answer not in options -> 400	**PASS**
Privacy	Other instructor sees shared, not private	**PASS**
Privacy	?visibility=PRIVATE cannot reveal another instructor's questions	**PASS**

Area	Check	Result
Privacy	Other instructor GET / approve / reject a private question -> 404	**PASS**
Workflow	Approve records approver + time; re-approve -> 409	**PASS**
Workflow	Owner rejects with reason; approving a rejected question -> 409	**PASS**
Bank	Bank contains only approved; ignores a status override in the query	**PASS**
Filtering	Filter by status + difficulty returns exactly the matching question	**PASS**

Rows group related assertions; the script reports 36 individual checks. They will be ported to a permanent automated suite (Vitest, isolated test database) in sub-step 5.

12. Engineering log, risks and open questions

12.1 Problems encountered and how they were resolved

Problem	Root cause	Resolution
npm ENOTEMPTY during install	Dependency install started while create-next-app's own install was still writing node_modules	Waited for all npm processes to exit, then reinstalled sequentially
Prisma P1010 'access denied'	A Homebrew PostgreSQL already listened on localhost:5432, so connections hit it instead of the container	Mapped the container to host port 5434 (5433 used by another project), configurable via DB_PORT
'docker compose' unknown	Machine has the standalone docker-compose binary (Colima), not the CLI plugin	npm scripts use docker-compose
.env.example ignored by git	Default Next.js .gitignore ignores .env*	Added !.env.example exception
prisma migrate dev refused the enum change	Removing an enum value that existing rows use needs interactive confirmation	Wrote the migration by hand (Section 4.6), applied with migrate deploy, verified with migrate diff

12.2 Known limitations (to address in later sub-steps)

- **Difficulty range is enforced in the application layer only.** A database CHECK constraint (difficulty BETWEEN 1 AND 5) should be added for defence in depth.
- **No rate limiting** on /auth/login and /auth/register; brute-force protection is needed before public deployment.
- **Tokens cannot be revoked early.** Mitigated by the per-request user reload (deleted users apply immediately); a token denylist or short-lived access + refresh tokens can follow.
- **A single shared invite code** gates registration; per-person invites or an admin approval step would allow revoking one person's invite.
- **options JSON is validated by Zod, not by the database.** Acceptable for now; could move to a dedicated table if per-option analytics are needed.

12.3 Open design questions

- Should an instructor be allowed to approve their **own shared** question, or should shared questions require a second instructor (four-eyes principle)?
- Should a rejected question be **editable and resubmittable** (REJECTED -> PENDING), and should edits to an approved question send it back for review?
- Should instructors be able to change visibility after approval (e.g. share a previously private question)?

13. Next steps

Sub-step	Deliverable
3 - Guard review	Audit every route against the permission matrix; DB CHECK constraint for difficulty; login rate limiting
4 - Seed data	prisma/seed.ts: several instructors, 2 curricula with chapters, questions in PENDING / APPROVED / REJECTED, private and shared
5 - Automated tests	Vitest suite on a separate test database covering the approval workflow and every privacy rule in Section 6
6 - Cloud readiness	Multi-stage Dockerfile (Next.js standalone output), docker-compose with app + db, README
Later steps	AI question generation (source = AI, still PENDING until approved); instructor-only AI exam generation by curriculum, chapters, difficulty and total duration